

Group 03 - Project Report

Advanced Analytics and Applications

Team 03



Authors:

Angela Feng (7397232)

Moritz Lang (7430881)

Beate Kranz (7435471)

Lennart Jekel (7444132)

Contact: ljekel1@smail.uni-koeln.de

Supervisor: Univ.-Prof. Dr. Wolfgang Ketter

Co-Supervisor: Janik Muires

Chair of Information Systems for Sustainable Society
Faculty of Management, Economics and Social Sciences
University of Cologne

2026-07-26

Table of contents

1 Problem Description	1
2 Data Description	1
3 Data Preparation	1
3.1 Summary of the Central Data-Preparation Steps	2
4 Data Analysis NN	3
5 SVM	5
5.1 SVR	6
5.2 Results	6
5.3 SVC	6
5.4 Results	6
5.5 How spatial resolution seems to change the outcome?	6
5.6 How temporal resolution changed the outcome?	6
5.7 Further Improvement	7
6 Spatial Encoding	7
7 SVR	7
8 SVC	7
9 Comparison	7
10 Smart Charging Reinforcement Learning	7
10.1 Motivation and Setup	7
10.2 Results	8
10.3 Implications	8
11 Conclusion	9
12 Appendix	9
12.1 Neural-network baseline training configuration	9
12.2 Neural-network baseline validation results	10
12.3 Trivial baseline test performance	10
12.4 Neural-network baseline diagnostics	11
12.5 Final census-tract 1h test scores	11
12.6 Final model test MAE by spatial unit	12
12.7 Final model skill scores against baselines	12
12.8 Neural-network grid search	12
12.9 Grid-search winner training history	13
13 Citations and References	13

List of Figures

1 Validation diagnostics of the census-tract 1h neural-network baseline: actual versus predicted demand (left), and MAE and average actual demand by demand bucket (right)	11
--	----

2	Training history of the census-tract 1h neural-network baseline. The dashed line marks the 20-epoch budget considered for the subsequent grid search.	11
3	Validation-MAE histories of all nine neural-network baseline models. Columns represent the temporal aggregation and rows the spatial unit; black markers identify the selected checkpoint epochs.	14
4	Training loss and validation Balanced MAE of the final census-tract 1h neural network. The selected checkpoint is marked in the validation history.	14
5	Test MAE of the final 1h models by spatial unit. All panels use the same nonlinear color scale and value range; the color-bar values remain MAE in trips per hour. .	15
6	Final-model test MAE Skill Scores against the zero, global-median, and spatial $\times$ time $\times$ weekday baselines. Positive values indicate that the neural network has a lower test MAE than the respective baseline. The broken x-axis preserves the extreme negative score of the unfiltered 24h model while keeping the other scores readable.	15
7	Training loss and validation Balanced MAE of the selected Stage-2 grid-search model	16

1 Problem Description

For operators of ride-hailing services tactical and strategic capabilities in operations management are a precondition for market success. On the one hand the operator needs to understand the dynamics of the market to make guided strategic decisions, like in which cities and areas a ride-hailing business is likely profitable. On the other hand it is also crucial for the operator to be able to make daily tactical decision, for example to predict where and when demand will be high.

We have conducted an analysis of the taxi demand in Chicago, US to enable our customer to build the tactical and strategic capabilities. These capabilities involve business and data understanding, whereby data understanding can be seen as the foundation. Therefore the data mining goal is to describe the dynamics of the taxi market in Chicago and develop prediction models to forecast demand. Further we evaluate whether these models are sufficient to make demand forecasting for daily operations. The business goal is to argue whether a market entry in Chicago looks promising to our customer. Within this report we present the findings of our analysis.

- Reinforcement Learning?
- Some real number from the taxi market (but look at data description first)

2 Data Description

Analyze taxi demand patterns: Specifically show how these patterns (start time, trip length, start and end location, price, average idle time between trips, and so on) for the given sample varies in different spatio-temporal resolution (i.e., census tract vs. varying hexagon diameter and/or temporal bin sizes). Give possible reasons for the observed patterns. Furthermore, can you identify spatial hot spots for trip demand using Gaussian Mixture Models (i.e., using Spatial Kernel Density Estimation)?

3 Data Preparation

Task: “ “ ” in other words, your method should predict for each spatial unit (in whatever spatial layer of analysis you decide on, e.g., hexagon or census tract) and time-basket (e.g., 08am-11.59am) the taxi demand. Important: It is not your goal to predict hour $x+1$ demand given hour x demand, rather to produce an answer to questions like “what demand can we expect on a rainy Sunday from 10 – 11 am around the outskirts of the city”. Also advise a reasonable validation strategy for your prediction model (i.e., definition of test, training data etc). “ “ ”

- Data sources: Taxi data, weather data, Census tracts, community areas, Point of interest data
- Taxi data was downloaded from the Chicago Data Portal via API
- Full run is from first january 2024 to first may 2026

- The sample run uses one reproducible, contiguous 14-day window (2026-04-13 through 2026-04-26, inclusive). This preserves complete weekdays, weekends, and daily demand cycles while keeping notebook runtimes short. A 400,000-trip cap is only a safety guard; reaching it is treated as an invalid, potentially truncated sample.
- Initially a data profiling was conducted to better understand the data, find null values and plausibility issues and coverage of the Data
- This data profiling layed down the basis for the following data-prep steps
- Data was prepared following a simple bronze-silver-gold pipeline logic, whereby each dataset runs through the levels:
- Bronze represents the raw data stored as parquet file
- Silver adjusted the data quality, missing values, inadequate data, consistency checks
- Gold aggregates data, joins different data sets
- Regarding gold / generating of the temporal-spatial dataset
- First we created a dataframe containing the temporal unit, i. e. 1h, 2h or 4h. This dataframe contained nothing but the timestamps
- We integrated the weather data to this dataframe and also holidays
- Next we merged the temporal dataframe with the spatial units, i. e. census tracts, hexagons and community areas
- This gave us the keys of the final dataset and we could aggregate the taxi data into this dataframe

Thereby creating certain data characteristics that become problematic and need to be addressed during predictive tasks: - Most Temporal-Spatial units contain no or only very small demand (e. g. 3 am in some outer area of chicago) while only few contain high demand - Therefore we decided to use community area as main spatial unit

3.1 Summary of the Central Data-Preparation Steps

- Load the taxi, weather, spatial-boundary, and OpenStreetMap point-of-interest data and store the raw inputs as Parquet files in the bronze layer.
- Profile the raw data for missing values, duplicates, temporal coverage, and implausible trip, fare, and location values; use these findings to define the subsequent cleaning rules.
- Create the silver taxi dataset by removing duplicate or implausible trips, requiring valid pickup information, and retaining trips with positive fares and totals, non-negative cost components, distances of more than 0 and at most 250 miles, and durations between 1 minute and 3 hours.
- Restrict the weather observations to the taxi-data period, repair the census-tract and community-area geometries, standardize their identifiers, and generate an H3 grid covering Chicago.
- Build the gold layer by assigning taxi trips and POIs to H3 cells, census tracts, and community areas and by excluding pickup locations outside the Chicago grid.

- Construct complete 1-, 2-, and 4-hour time grids and enrich them with aggregated and imputed weather data, Illinois holiday indicators, and cyclical month, weekday, and hour features.
- Cross the temporal grids with every spatial unit, add POI counts by category, and aggregate taxi pickups as `trip_count`; explicitly retain spatial-temporal combinations with zero demand.
- Produce comparable demand datasets for all temporal and spatial resolutions, with community areas used as the principal modeling unit because finer resolutions contain a particularly high proportion of sparse or zero-demand observations.
- Create reproducible train, validation, and test datasets (approximately 70/15/15), grouping complete calendar days into the same split and checking that no dates leak across the partitions.
- 4 Datasets in silver:
 - Community area: complete data set as community area is always as spatial unit given
 - Census tracts: is more granular than community area and not always given (number of missing values), therefore we remove data without `census_tract` here
 - Hexagon: the hexagons are build on the `census_tracts` and therefore only reasonable when we have `census_tract` data
 - community area filtered: to make a fair comparision between the spatial units we create a filtered dataset of community area => Thereby we have one main dataset on community area and for a fair comparision of spatial units we have 3 filtered datasets

4 Data Analysis NN

- Baseline prediction
 - Zero prediction
 - Avg Spatial x Time x Weekday
 - Global median
 - see Table 4
- General validation strategy:
 - Test only for final model and comparison SVM vs
- Target and Features
 - We selected “`trip_count`” as target for the demand, as this will tell us how many taxi trips were started in one spatial time unit
 - Features: cyclic encoding of hour, weekday and month; weather data; holiday flag and POI information; all other taxi related columns were removed as it would otherwise lead to leakage
- Preprocessing steps:
 - Spatial Encoding is needed as the spatial units are otherwise interpreted as continuous feature
 - StandardScaler for the `feature_cols` fitted on training data only

In this section we describe the approach of developing and evaluating a neural network as prediction model. Our approach is divided into three sequential steps. First we developed a simple and reproduceable baseline NN, second we performed a grid search based on the findings of the baseline NN and lastly we trained a final model based on the grid search results.

- The baseline model allow us to make a first guess whether NN's are a suitable prediction model for this use case and give us an intuition of what performance we can expect. With the grid search we can compare different models and select the best performing one. The goal of the grid search is not to develop a final model but to make decisions about the architecture and training parameters. Therefore we need a final training in the last step.

The baseline NN is trained for all spatial-units and time buckets, thereby allowing a fair comparison between them. Within this development phase our main goal was to determine whether a NN can generally perform better compared to the baseline and which spatial-time-unit is most suited. The architecture of the baseline NN is kept simple with two hidden layers (64 and 32), ReLu activation function and a spatial encoding for the categorical spatial units. A spatial encoding is necessary, because the spatial ID is a categorical value. The numerical features were scaled using the StandardScaler, which was fitted only on the train set. After this scaling the features have a mean close to 0 and variance close to 1. This is helpful as distances between values of features are standardized. In Table 1 and Table 2 we present the architecture and trainings configuration. The checkpoint-selection metric, which determines the best model throughout the epochs, is MAE on the validation set. We decided to use MAE, because it is simple and very good interpretable. However, MAE is not suitable for comparisons between different spatial-time-units. Therefore, to compare different spatial-time-units we calculate a MAE Skill Score, which is $1 - \text{val MAE} / \text{baseline MAE}$, where we selected the mean spatial x time x weekday as baseline. This means that a MAE Skill Score of 0 means no improvement compared to the baseline predictions, while a positive values shows an improvement. In Table 3 the results of the baseline NN are shown. The spatial-time-unit `census_tract_1h` is the only dataset with a positive MAE skill score, however the score is also very close to zero with 0.0043. From the table it is Further observable that the datasets on 24h time bucket are performing worst. Notably all datasets despite `community_area_24h` outperform the always zero prediction. The worse performance of 24h may be due to the significant lower training size of the 24h datasets. We decided to have a closer look at the best performing model for `census_tract_1h` and the plots in Figure 1 show that the model has a particular weakness for predicting higher demand.

- Show how you model's performance varies as you increase or decrease temporal or spatial resolution

Figure 3

Therefore we decided that the main focus for the next NN should be to distinguish between zero and real demand. A first step was to switch the checkpoint-selection metric from MAE to Balanced MAE, where Balanced MAE is $0,5 * \text{non-zero MAE} + 0,5 * \text{zero-MAE}$, thereby trying to ensure that zero-demand is not entirely driving the training. The second step was to perform a grid search in two stages. The first stage was exploring two strategies of dealing with the problem of zero-demand: undersampling and weighted training. The search space contains for each strategy three settings, which are shown in Table 6. In the first stage of the grid search the architecture was fixed to the architecture of the baseline and each model was trained for 20 epochs. We considered 20 epochs to be sufficient as it could be observed in the trainings history of the baseline model for `census_tract_1h` that the loss slows down decreasing at around 7 to 12 epochs (see Figure 2). The results of the first stage are shown in Table 8. Notably all models perform worse than the spatial x time x weekday baseline, as all models have a negative MAE skill score. But this may be due to the limited trainings epochs as all runs have their best epoch

very close to 20. Despite it is evidence that weighting is performing better than undersampling. All three runs with weights have higher MAE skill scores than the ones with undersampling. Therefore weighted loss is selected as strategy and used in the second stage of the grid search, where we tested different model architectures and learning rates. In Table 7 the search space of the second stage is shown. The results of the second stage as shown in Table 8 show again mainly negative MAE skill scores and best epochs close to the maximum number of epochs. The model S2-7 with the hidden layers 128-64, learning rate of 0.001 and a weight of 5 for positive demand performed best with a score of 0.0014 and balanced MAE of 2.7877. The best balanced MAE was achieved in epoch 29 of 30 (see Figure 7), which highlights that further training with more epochs may increase the performance.

The final model is therefore trained on the architecture and trainings configuration of the model S2-7 but with 100 epochs and a patience of 10. Despite giving a maximum of 100 epochs the best model was trained in epoch 26 (see Figure 4) with a balanced MAE of 2.8775 and spatial x time x weekday skill score of 0.0253. The final model is therefore slightly better than the baseline and the NN baseline. The final model has a zero-demand MAE of 0.0123 and a non-zero demand MAE of 5.7427. The large gap between these two MAE's shows that also for the final model the main difficulty is that zero-demand dominates the training and the model struggles to predict higher demand. This is also evident from the fact that the peak-demand MAE is with 17.1399 much higher than the non-zero-demand MAE and that the recall with 0.7162 is lower than the precision with 0.8123. The model struggles to predict real demand, but if it does so it is more certain. All the test-set performance metric are shown in Table 5.

The fact that the recall is lower than the precision also leads us to propose improvements for a follow-up project. We recommend that prior to the regressions predicts a classifier model should predict the probability of low or high demand. Further we recommend exploring separate regression models for low and high demand. The final prediction could then be an ensemble of the classification, low and high demand prediction, e. g. final prediction = $p_{\text{high}} * \text{high_model_pred} + (1 - p_{\text{high}}) * \text{low_model_pred}$. A further lever may be to include lag-features like the demand in the last week or day. Then it would no longer be recommended to perform a random split of the train, validation and test data, but instead a chronological split should be used.

Despite the grid search was conducted only on the census_tract_1h dataset, we decided to train the final model S2-7 on all datasets. Of course the comparison of the final model performance between the dataset is therefore not completely fair.

- @fig-final-census-tract-test-mae
- @fig-final-mae-skill-trivial-baselines

5 SVM

To understand demand in different areas of the city, we used a neural network and SVM. To be able to compare the two models, we focused on SVM with regression. The target here, as above, is the trip demand specifically the trip count.

set non negatives to 0 since there cant be negative demand

For SVM we used the same data as described above. With time units consisting of one hour, four hour and twentyfour hour. The spatial units used include hexagons?? The target was defined as "trip_count". The amount of started trips during The features are Features: cyclic encoding of hour, weekday and month; weather data; holiday flag and POI information; all other taxi related columns were removed as it would otherwise lead to leakage and the spatial

unit encoded as latitude longitude with x, y, z due to show the dependencies of latitude and longitude. This was chosen since most other encoding did not work for all three spatial units. Especially OneHot encoding only worked for community areas. We started with a simple kernel without anything. At first we added

To compare the results with the Neural Network we decided to focus on regression instead of classification. The biggest problem we run into during the development is the runtime of the gridsearch and the fitting. These exceeded multiple hours depending on the spatial unit and time frame.

The changes we made included: - changes we made during the development of th svm in steps and why we decided on them - started with a basic svr on rbf kernel which is the basic kernel for scikits() implementation of svms - added grid search to find best hyperparameter - added scaler - changed grid search to only include the hyperparameter relevant for each kernel - added nystroem - changed gridsearchcv to randomsearchcv to make hyperparameter fitting quicker - changed gridsearch to include standardscaler in pipeline instead of manually - added transgressor - added memory - for linear use linear - change available hyperparamter through testing - changed to halving... instead of randomsearchcv due to better results - tested this out as the last option due to still being experimental

The last version consisted of a pipeline which is: TransformedTargetRegressor, standard-scaler, Nystroem and

- due to the data having a lot of y=0 the grid search complains about convergance issues -> this is another reason why svm for this particaluar data with h3 is not the best method
- The time frames and spatial units, used above in the Neural Network() were used here as well.
- These include time frames of one, two and twentyfour hours.
- The used spatial units include

5.1 SVR

5.2 Results

Overall the spatial units which have bigger units outside the center of the city where most people take taxis seem to perform better. This might be due to the fact that the grid searcha as well as the fitting does not

5.3 SVC

The classification was developed at the side and run on the best SVRs.

5.4 Results

5.5 How spatial resolution seems to change the outcome?

Generally

5.6 How temporal resoltion changed the outcome?

Mid temporal resoltion seems to be the best so 4h or sometimes 1 seems to overall be the best performing

5.7 Further Improvement

The first improvement we would recommend it to use data, that does not consist of a lot of trip count zero. This lead to bad convergance during the grid search as well as the model fitting. The next time we would also recommend to directly develop the model to run on GPU due to the significant amount of data. But overall we would not recommend to use SVMs for this kind of data especially for classification there are models which can handle more similar data better.

SVM

6 Spatial Encoding

7 SVR

8 SVC

9 Comparison

10 Smart Charging Reinforcement Learning

10.1 Motivation and Setup

The daily operation of electric vehicles requires them to recharge daily. Depending on the time of day, charging costs can vary significantly, especially between peak and off-peak hours. Therefore, charging cost is a significant operational expense for an electric fleet.

A naive strategy of always charging at maximum power ensures the battery is full, but comes at an unnecessarily high cost since the cost grows exponentially with power. However, if we assume the vehicle is home for a fixed window, for example between 2pm and 4pm, we can be smart about when and how much to charge. With reinforcement learning we can simulate this smart charging behaviour, where an agent learns to balance cost savings against the risk of running out of energy.

We model this as a Markov Decision Process (MDP), where the agent observes the current time and battery level, picks a charging power level, and receives feedback in the form of a cost signal.

Our MDP is defined as follows:

- State: the current 15-minute timestep (2:00pm, 2:15pm, ..., 4pm) and the current battery level.
- Action: one of 4 discrete charging power levels: 0 kW (off), 7 kW (low), 14 kW (medium), 22 kW (high).
- Transition: the battery increases by the chosen power $\times 0.25\text{h}$, capped at battery capacity.
- Reward: negative charging cost each step. At the final step (4pm), a random energy demand is drawn from a normal distribution ($\mu = 30 \text{ kWh}$, $\sigma = 5 \text{ kWh}$). If the battery cannot cover this demand, a large penalty is applied.

We made the following assumptions, grounded in real-world data:

- Battery capacity: 60 kWh; Tesla Model Y is one of the most used EV taxi models in North America, and 60 kWh represents a typical usable battery capacity for current EVs.

- Charging levels: 0, 7, 14, 22 kW; Level 2 home chargers support up to 22 kW, which is sufficient for the Tesla Model Y.
- Starting battery at 2pm: random between 5 and 20 kWh; the vehicle arrives partially depleted after a morning shift.
- Cost coefficients: 0.052–0.076; calibrated to match real Chicago home charging rates of \$0.11–\$0.16 per kWh, with the coefficient increasing linearly across timesteps to reflect time-of-use pricing.

We solve this using Q-Learning, a standard reinforcement learning method that learns optimal decisions through trial and error. Q-Learning is well suited here because the state space is small and discrete, allowing the agent to learn a complete policy without the overhead of a neural network.

We additionally implemented a Deep Q-Network (DQN) to test whether a neural network-based approach could match or improve on the tabular solution. While Q-Learning works well for our small state space, DQN would scale better to more complex scenarios, for example if the charging window were longer or if real-time electricity prices were included as an additional state variable.

10.2 Results

We evaluated the learned policies over 20000 simulated episodes against a naive baseline that always charges at maximum power (22kW). The results are summarized below:

	Avg Cost Savings	Avg Battery	Fail Rate
Baseline (always max)	10.14	0.0%	55.9
Q-Learning	6.38	37.1%	47.1
DQN	8.24	18.8%	50.2

The Q-Learning agent achieves a 37.1% cost reduction compared to the baseline while maintaining a 0% out-of-energy rate. It does this by charging more aggressively during the early, cheaper timesteps and reducing or skipping charging later when cost coefficients are higher. The average battery departure (47.1 kWh) comfortably exceeds the mean demand of 30 kWh, providing a safety margin against demand uncertainty without overcharging.

The DQN agent also outperforms the baseline with an 18.8% cost reduction, though it charges more conservatively than Q-Learning, resulting in a higher average battery at departures (50.2kWh). This is expected since the tabular Q-Learning approach can represent the optimal policy exactly for our small, discrete state space. Meanwhile the DQN introduces approximation, however, its advantage lies in its scalability, as it could handle a more complex scenario with continuous state variables or a longer charging window without modification.

All three policies achieve a 0% out of energy rate, confirming that the penalty mechanism successfully teaches the agents to prioritize energy sufficiency. This is substantial, since this could leave the driver stranded and lose a full day of revenue.

10.3 Implications

The results demonstrate that even in a simplified two-hour charging window, an intelligent charging agent can reduce costs by over a third compared to a naive strategy. Scaled to an entire fleet, these savings become significant. For example, if a fleet of 100 vehicles each charges once per day, a 37% reduction in per-vehicle charging cost translates directly into lower daily operating expenses.

The smart charging system is fully automated and requires no manual intervention from drivers or dispatchers. Once trained, the agent makes real-time charging decisions based on the current battery level and time of day, adapting naturally to varying starting conditions.

Moreover, the approach is flexible. If the electricity pricing structures change or the charging window shifts or the parameters are changed, the agent can simply be retrained on the updated parameters. This makes it a future-proof component of the fleet’s operational toolkit, complementing the demand prediction models developed in earlier sections of this analysis.

11 Conclusion

Discuss the implications of your results for the potential fleet operator (client).

Which further analysis would you consider useful and could be conducted on the given dataset?

Which other external data sources might be interesting to consider? Is it sensible to install private charging stations, or should the operator draw on public charging stations?

- Data mining goal
- Business goal

12 Appendix

Table 1: Architecture of the neural-network baseline

Component	Input dimension	Output dimension	Activation
Numeric input	n_{features}	n_{features}	–
Hidden layer 1	n_{features}	64	ReLU
Hidden layer 2	64	32	ReLU
Numeric output	32	1	Linear
Spatial bias	Spatial-unit ID	1	–
Combination	Numeric score + spatial bias		1 Addition
Final output	1	1	Softplus

The spatial bias is implemented as a one-dimensional embedding. It represents an additive spatial intercept rather than a multidimensional representation-learning embedding. Index zero is reserved for spatial units not observed in the training split and has no learned effect.

12.1 Neural-network baseline training configuration

Table 2: Training configuration of the neural-network baseline

Setting	Configuration
Datasets	H3 resolution 7, census tracts, and filtered community areas
Temporal units	1h, 4h, and 24h
Number of models	9
Training objective	Mean squared error (MSE)

Setting	Configuration
Optimizer	Adam
Learning rate	0.001
Batch size	1,024
Maximum epochs	100
Early-stopping patience	10 epochs
Minimum improvement	0
Checkpoint-selection metric	Validation MAE
Output constraint	Softplus activation
Numerical preprocessing	StandardScaler fitted exclusively on TRAIN
Spatial preprocessing	Categorical mapping fitted exclusively on TRAIN
Reproducibility	Dataset-specific seed, starting at 42
Model development split	TRAIN for fitting and VAL for model selection
Final evaluation split	TEST remains unused during model development

12.2 Neural-network baseline validation results

Table 3: Validation results of the neural-network baseline models

	Model	NN MAE	S×T×W MAE	Skill: zero	Skill: median	Skill: S×T×W
1	census_tracts_1h	0.1396	0.1402	0.6187	0.6187	0.0043
2	community_areas_1h	1.3758	1.2853	0.6704	0.6704	-0.0705
8	community_areas_4h	5.4196	4.6133	0.6754	0.6754	-0.1748
7	census_tracts_4h	0.5584	0.4659	0.6186	0.6186	-0.1985
0	hexagon_h3r7_1h	0.7556	0.6220	0.6168	0.6168	-0.2148
6	hexagon_h3r7_4h	2.9352	2.2093	0.6278	0.6278	-0.3286
3	hexagon_h3r7_24h	15.0754	11.1707	0.6814	0.6814	-0.3495
4	census_tracts_24h	3.3413	2.2846	0.6197	0.6197	-0.4625
5	community_areas_24h	100.9842	23.6039	-0.0081	-0.0081	-3.2783

12.3 Trivial baseline test performance

Table 4: Test MAE of the three trivial baselines for the filtered datasets

Dataset	Zero MAE	Global median MAE	Spatial-time-weekday MAE
hexagon_h3r7_1h	1.9365	1.9365	0.5924
census_tracts_1h	0.3595	0.3595	0.1356
community_areas_1h	4.0993	4.0993	1.2164
hexagon_h3r7_4h	7.7459	7.7459	2.0454
census_tracts_4h	1.438	1.438	0.4397
community_areas_4h	16.3973	16.3973	4.2469
hexagon_h3r7_24h	46.4757	46.4757	10.442
census_tracts_24h	8.6282	8.6282	2.1754
community_areas_24h	98.3836	98.3836	21.9823

12.4 Neural-network baseline diagnostics

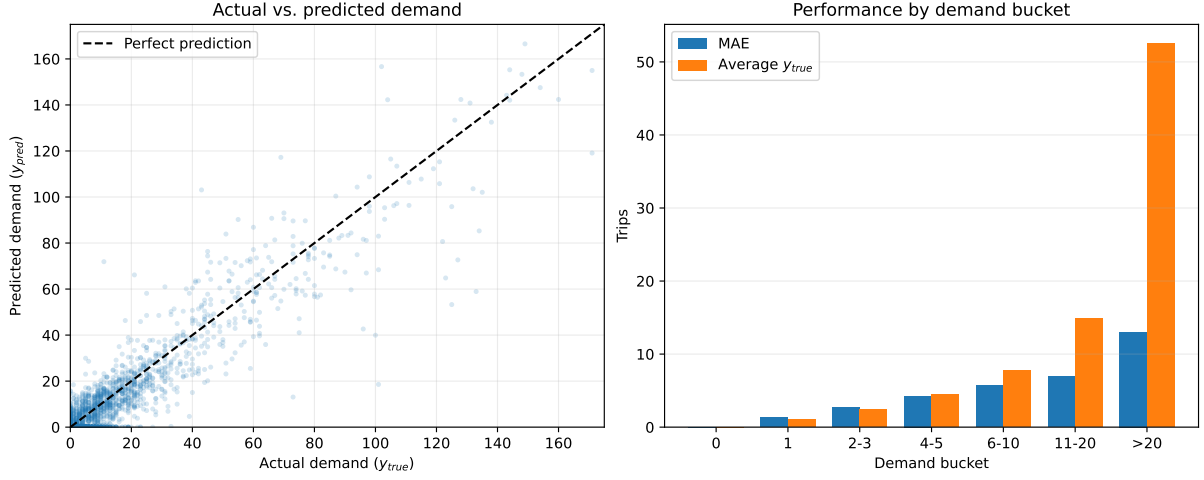


Figure 1: Validation diagnostics of the census-tract 1h neural-network baseline: actual versus predicted demand (left), and MAE and average actual demand by demand bucket (right)

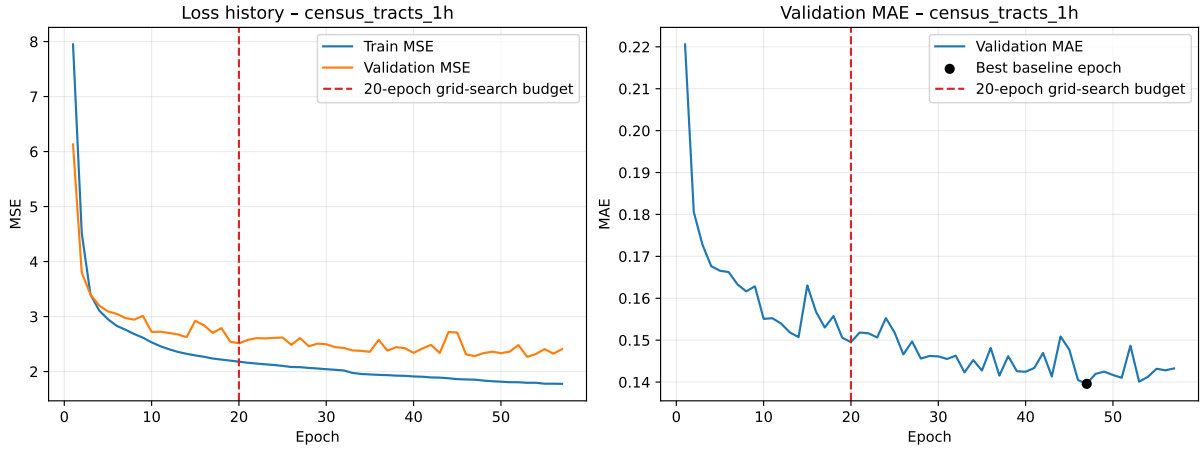


Figure 2: Training history of the census-tract 1h neural-network baseline. The dashed line marks the 20-epoch budget considered for the subsequent grid search.

12.5 Final census-tract 1h test scores

Table 5: Test-set performance of the final census-tract 1h neural network

Metric	Value
MAE	0.1322
RMSE	1.4994
R^2	0.8939
Zero-demand MAE	0.0123
Non-zero-demand MAE	5.7427
Peak-demand MAE	17.1399
Balanced MAE	2.8775
MAE Skill Score vs. zero	0.6322

Table 5: Test-set performance of the final census-tract 1h neural network

Metric	Value
MAE Skill Score vs. global median	0.6322
MAE Skill Score vs. spatial $\times$ time $\times$ weekday	0.0253
Demand precision	0.8123
Demand recall	0.7162

12.6 Final model test MAE by spatial unit

12.7 Final model skill scores against baselines

12.8 Neural-network grid search

Table 6: Models evaluated in Stage 1 of the neural-network grid search

Model	Strategy	Strategy parameter
S1-1	standard	–
S1-2	weighted_loss	positive weight = 2
S1-3	weighted_loss	positive weight = 5
S1-4	weighted_loss	positive weight = 10
S1-5	zero_undersampling	zero:positive = 1
S1-6	zero_undersampling	zero:positive = 2
S1-7	zero_undersampling	zero:positive = 5

Table 7: Models evaluated in Stage 2 of the neural-network grid search

Model	Positive weight	Hidden layers	Learning rate
S2-1	2.0	64-32	0.001
S2-2	2.0	64-32	0.0003
S2-3	2.0	128-64	0.001
S2-4	2.0	128-64	0.0003
S2-5	5.0	64-32	0.001
S2-6	5.0	64-32	0.0003
S2-7	5.0	128-64	0.001
S2-8	5.0	128-64	0.0003

Table 8: Validation performance of all Stage-1 and Stage-2 grid-search models

Stage	Model	Best epoch	Balanced MAE	MAE skill score
1	S1-1	20	3.4055	-0.1528
1	S1-2	19	3.1548	-0.0242
1	S1-3	20	3.1001	-0.0367
1	S1-4	15	3.018	-0.0797
1	S1-5	19	3.66	-0.9906
1	S1-6	19	3.6847	-0.4227

Table 8: Validation performance of all Stage-1 and Stage-2 grid-search models

Stage	Model	Best epoch	Balanced MAE	MAE skill score
1	S1-7	20	3.5237	-0.2444
2	S2-1	24	3.0929	-0.0001
2	S2-2	29	3.3638	-0.0729
2	S2-3	16	3.0774	-0.0014
2	S2-4	29	3.4619	-0.0997
2	S2-5	29	3.0245	-0.1996
2	S2-6	29	3.3499	-0.0885
2	S2-7	29	2.7877	0.0014
2	S2-8	24	3.422	-0.104

12.9 Grid-search winner training history

13 Citations and References

You can easily cite your sources using BibLaTeX. For example, you can cite the Quarto documentation (Team, 2024) or a textbook like “R for Data Science” (Wickham et al., 2023). The bibliography will be automatically generated at the end of the document.

References

- Team, Q. (2024). Quarto: An open-source scientific and technical publishing system. *Journal of Modern Data Science*. <https://quarto.org>
- Wickham, H., Çetinkaya-Rundel, M., & Golemund, G. (2023). *R for data science*. O’Reilly Media.

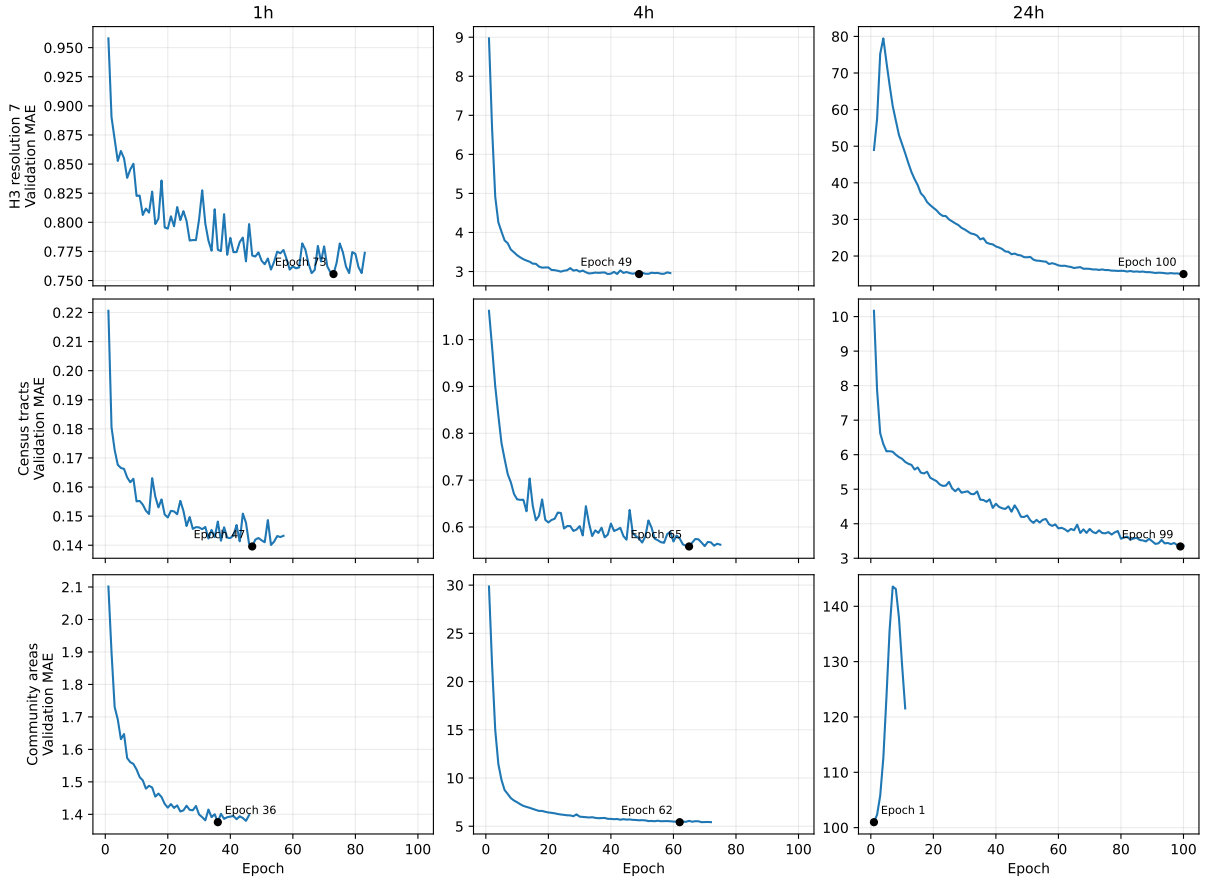


Figure 3: Validation-MAE histories of all nine neural-network baseline models. Columns represent the temporal aggregation and rows the spatial unit; black markers identify the selected checkpoint epochs.

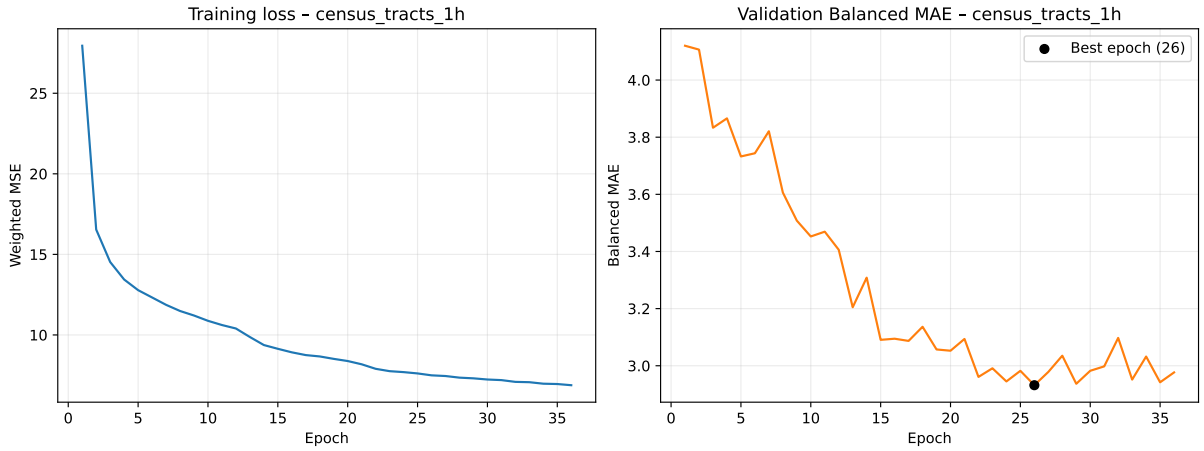


Figure 4: Training loss and validation Balanced MAE of the final census-tract 1h neural network. The selected checkpoint is marked in the validation history.

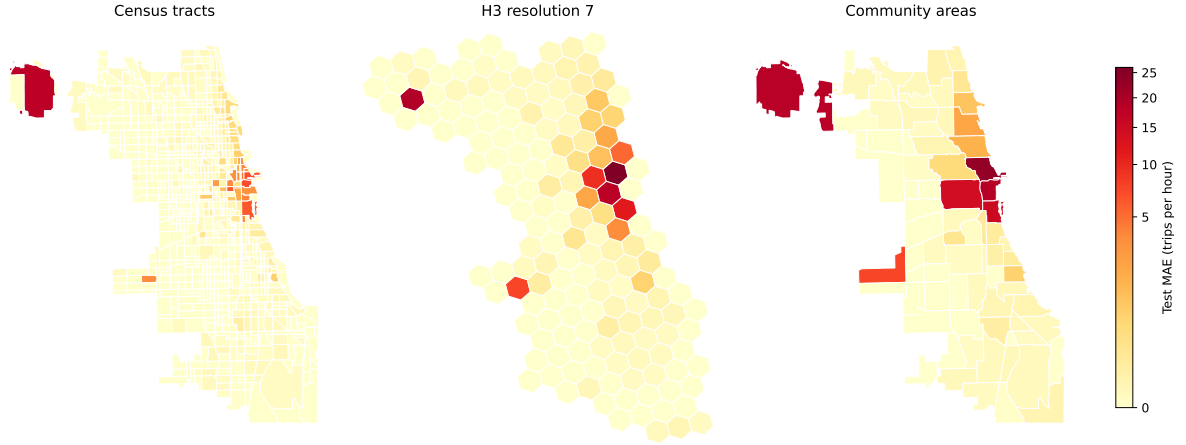


Figure 5: Test MAE of the final 1h models by spatial unit. All panels use the same nonlinear color scale and value range; the color-bar values remain MAE in trips per hour.

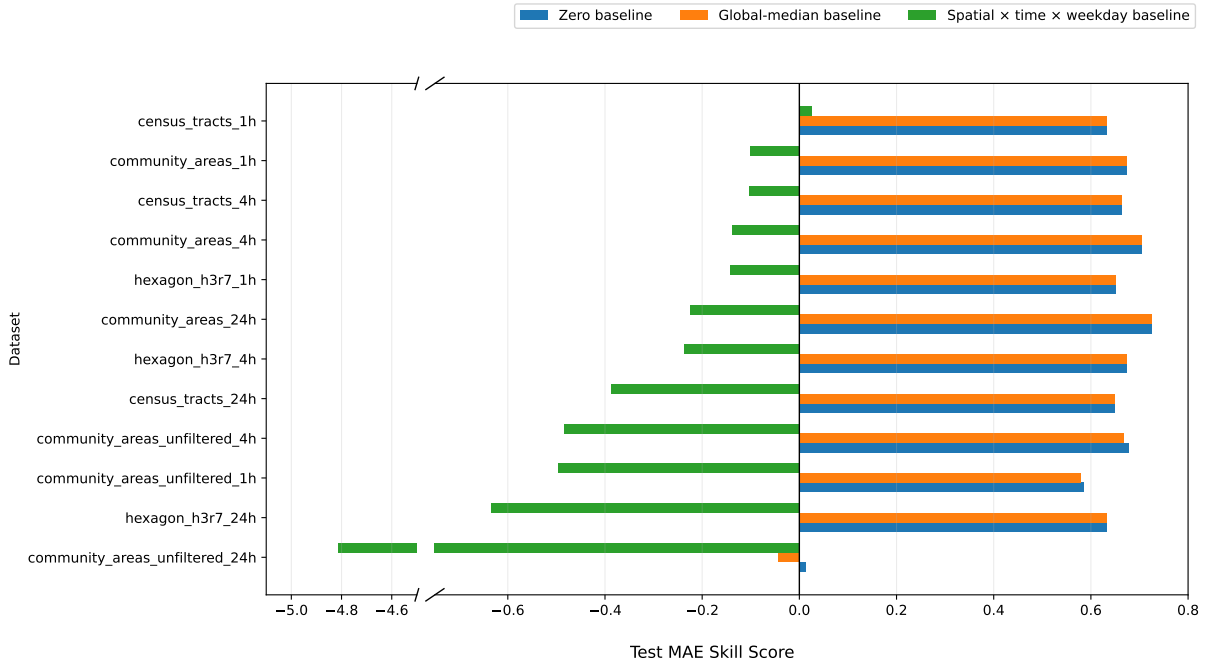


Figure 6: Final-model test MAE Skill Scores against the zero, global-median, and spatial $\times$ time $\times$ weekday baselines. Positive values indicate that the neural network has a lower test MAE than the respective baseline. The broken x-axis preserves the extreme negative score of the unfiltered 24h model while keeping the other scores readable.

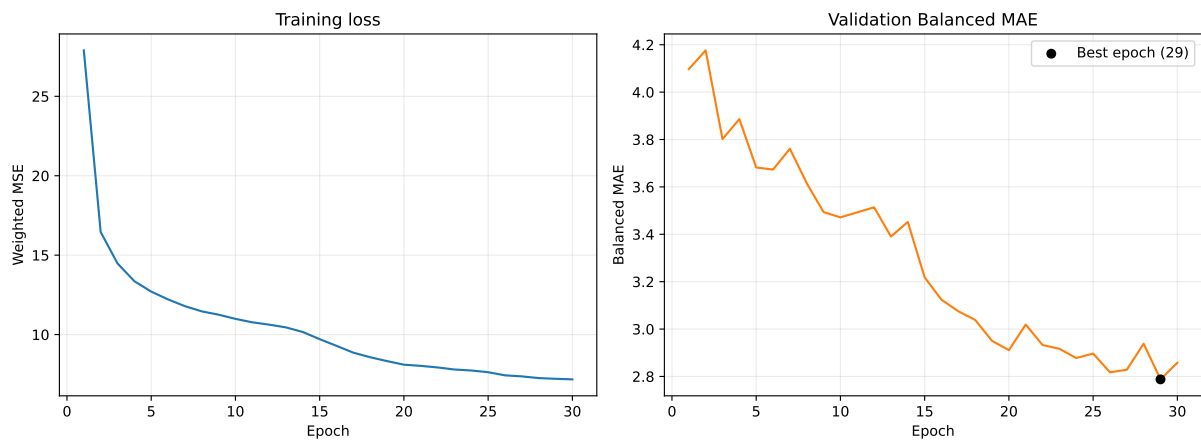


Figure 7: Training loss and validation Balanced MAE of the selected Stage-2 grid-search model